ASSIGNMENT-13.5

NAME: HABEEBA KHANAM

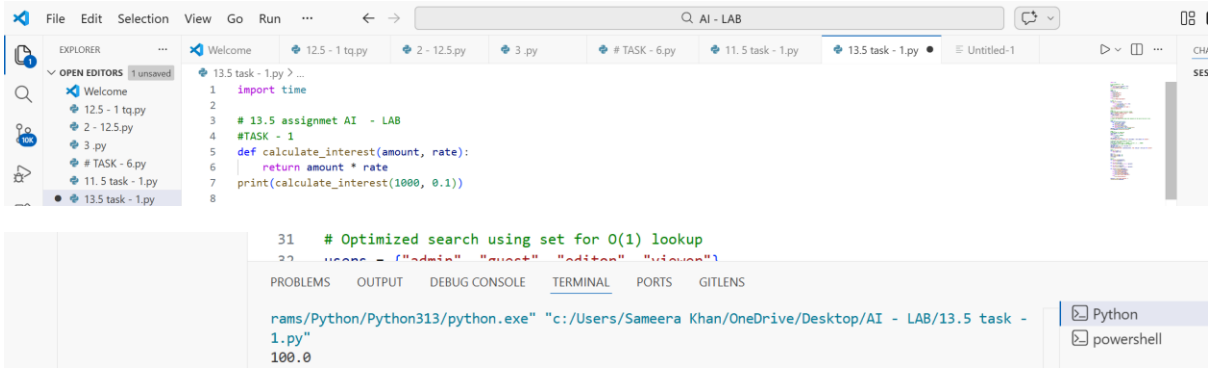
HALL TICKET :2303A51474

BATCH NO:29

LAB 13: CODE REFACTORING – IMPROVING LEGACY CODE WITH AI SUGGESTIONS

TASK DESCRIPTION #1 (REFACTORING – REMOVING GLOBAL VARIABLES)

CODE:



OBSERVATION:

1. Removed global variable dependency
2. Function becomes reusable and testable
3. Better modularity

TASK DESCRIPTION #2 : (REFACTORING DEEPLY NESTED CONDITIONALS)

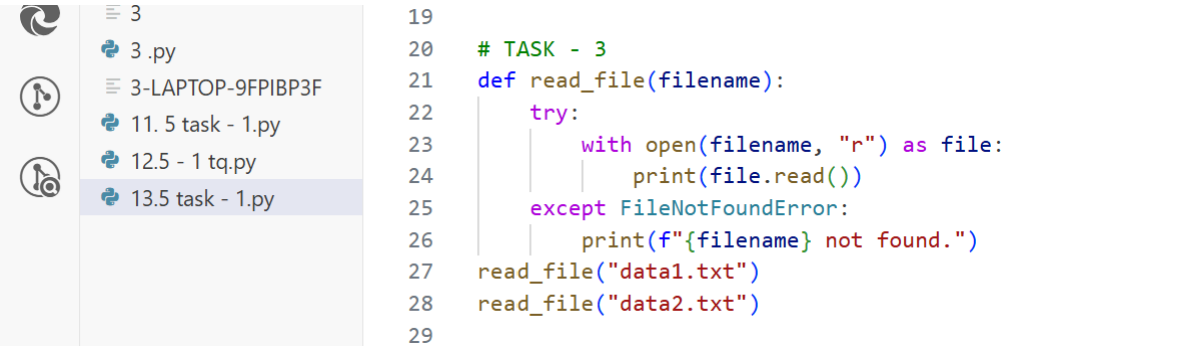
Code:

Observation:

1. Removed deep nesting
2. Improved readability
3. Easier to maintain

TASK 3 (REFACTORING REPEATED FILE HANDLING CODE)

CODE:



Observation:

1. Uses **context manager (with)**
2. Avoids **resource leaks**
3. Reusable function (DRY principle)

TASK 4 (OPTIMIZING SEARCH LOGIC)

CODE:

```python
#TASK - 4
# Optimized search using set for O(1) lookup
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
# Using a set reduces the average time complexity of the search from O(n) to O(1).
```

```
Enter username: anas
Access Denied
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/13.5 task - 1.py"
```

OBSERVATION:

1. Set lookup O(1) instead of list O(n)
2. Cleaner logic
3. Faster for large datasets

TASK 5 (REFACTORING PROCEDURAL CODE INTO OOP DESIGN)

CODE:

```python
#TASK - 5
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary
    def calculate_tax(self):
        return self.salary * 0.2
    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()
employee = EmployeeSalaryCalculator(50000)
print(employee.calculate_net_salary())
```

```
Enter username: sam
Access Denied
40000.0
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/13.5 task - 1.py"
```

OBSERVATION:

1. Encapsulation

2. Reusable design
3. Scalable for payroll systems

TASK 6 (REFACTORING FOR PERFORMANCE OPTIMIZATION)

CODE:

```python
#TASK - 6
# Legacy code for comparison
start_legacy = time.time()
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
end_legacy = time.time()
print(f"Legacy total: {total}, Time: {end_legacy - start_legacy:.6f} seconds")

# Optimized code using mathematical formula
start_opt = time.time()
# Sum of even numbers from 1 to 999999 is sum of 2, 4, ..., 999998
n = (999999) // 2  # number of even numbers
last_even = n * 2
optimized_total = n * (2 + last_even) // 2
end_opt = time.time()
print(f"Optimized total: {optimized_total}, Time: {end_opt - start_opt:.6f} seconds")
```

```
40000.0
Legacy total: 249999500000, Time: 0.079418 seconds
Optimized total: 249999500000, Time: 0.000007 seconds
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Prog
```

OBSERVATION:

1. **Mathematical formula → O(1)**
2. Loop method → **O(n)**
3. Much faster for large data

TASK 7 (REMOVING HIDDEN SIDE EFFECTS)

CODE:

```python
#Task - 7
def add_item(data, x):
    return data + [x]

data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

```
ot found.
Enter username: editor
Access Granted
40000.0
Legacy total: 249999500000, Time: 0.ot found.
Enter username: editor
Access Granted
40000.0
ot found.
```

OBSERVATION:

1. No mutation of global variables
2. Predictable functional behavior
3. Easier to test

TASK 8 (REFACTORING COMPLEX INPUT VALIDATION LOGIC)

CODE:

```python
#Task-8
def is_long_enough(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

def validate_password(password):
    if not is_long_enough(password):
        print("Password too short")
    elif not has_digit(password):
        print("Must contain digit")
    elif not has_uppercase(password):
        print("Must contain uppercase")
    else:
        print("Valid Password")

password = input("Enter password: ")
validate_password(password)
```

```
Legacy total: 249999500000, Time: 0.067443 seconds
Optimized total: 249999500000, Time: 0.000008 seconds
[10, 20]
Enter password: 1234
Password too short
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

1. Modular functions
2. Easier unit testing
3. Cleaner validation logic